if, if-else, else-if ladder, nested if, switch

1. Introduction to Conditional Statements

In Java, conditional statements are used to control the flow of execution based on certain conditions. These statements allow a program to make decisions and execute different blocks of code depending on whether a condition evaluates to true or false.

In real life, decision-making happens all the time:

- If it rains → take an umbrella
- If marks are above 90 → grade A
- If age ≥ 18 → eligible to vote

Similarly, Java uses conditional statements to implement such logic.

---

2. Types of Conditional Statements in Java

Java provides the following decision-making statements:

1. if statement
2. if-else statement
3. else-if ladder
4. nested if
5. switch statement

---

3. if Statement

Definition

The if statement is the most basic conditional statement. It executes a block of code only if the condition is true.

Syntax

```
if(condition){
   // code to execute
}
```

Working

- Condition is evaluated
- If true → code runs
- If false → skipped

Example

```
int number = 10;


if(number > 0){
   System.out.println("Number is positive");
}
```

Flow Explanation

1. Check if number > 0
2. If yes → print message
3. If no → do nothing

Important Notes

- Condition must return boolean (true/false)
- Curly braces {} are optional for single statement but recommended

4. if-else Statement

Definition

The if-else statement executes one block if condition is true and another if false.

Syntax

```
if(condition){
   // true block
} else {
   // false block
}
```

Example

```
int age = 16;


if(age >= 18){
   System.out.println("Eligible to vote");
} else {
   System.out.println("Not eligible");
}
```

Working

- Condition checked
- If true → if block executes
- If false → else block executes

Use Case

Used when there are only two possibilities.

---

5. else-if Ladder

Definition

Used when multiple conditions need to be checked.

Syntax

```
if(condition1){
   // block 1
} else if(condition2){
   // block 2
} else if(condition3){
   // block 3
} else {
   // default block
}
```

Example

```
int marks = 85;


if(marks >= 90){
   System.out.println("Grade A");
} else if(marks >= 70){
   System.out.println("Grade B");
} else if(marks >= 50){
   System.out.println("Grade C");
} else {
   System.out.println("Fail");
}
```

Working

- Conditions checked one by one
- First true condition executes
- Remaining conditions skipped

Important

Order matters! Always write conditions carefully.

---

6. Nested if Statement

Definition

An if inside another if.

Syntax

```
if(condition1){
   if(condition2){
      // inner block
   }
}
```

Example

```
int age = 20;
boolean hasLicense = true;

if(age >= 18){
   if(hasLicense){
      System.out.println("You can drive");
   } else {
      System.out.println("Get a license first");
   }
}
```

Use Case

When multiple dependent conditions exist.

---

7. switch Statement

Definition

Used to execute one block among many based on a variable value.

Syntax

```
switch(expression){
   case value1:
      // code
      break;
   case value2:
      // code
      break;
   default:
      // default code
}
```

Example

```
int day = 2;


switch(day){
   case 1:
      System.out.println("Monday");
      break;
   case 2:
      System.out.println("Tuesday");
      break;
   default:
```

```
    System.out.println("Invalid");
}
```

Working

- Expression evaluated
- Matching case executed
- break stops further execution

---

8. Difference: if vs switch

Feature	if	switch
Condition	Any boolean	Fixed values
Range	Yes	No
Data types	All	Limited
Flexibility	High	Medium

---

9. Logical Operators in Conditions

Used with conditional statements:

Operator	Meaning
&&	AND
!	NOT

Example

```
int age = 25;


if(age > 18 && age < 60){
   System.out.println("Working age");
}
```

10. Relational Operators

Operator	Meaning
==	equal
!=	not equal
>	greater
<	less
>=	greater equal
<=	less equal

11. Real-Life Examples

Example 1: Even/Odd

```
int num = 7;


if(num % 2 == 0){
   System.out.println("Even");
} else {
   System.out.println("Odd");
}
```

Example 2: Maximum of 3 numbers

```
int a = 10, b = 20, c = 15;


if(a > b && a > c){
   System.out.println("A is largest");
```

```
} else if(b > c){
   System.out.println("B is largest");
} else {
   System.out.println("C is largest");
}
```

---

12. Common Mistakes

❌ Using = instead of ==
❌ Missing break in switch
❌ Wrong order in else-if ladder
❌ Not using braces

---

13. Best Practices

✓ Always use braces {}
✓ Keep conditions simple
✓ Avoid deep nesting
✓ Use switch for fixed values

---

14. Interview Questions

1. Difference between if and switch?
2. What is nested if?
3. Can switch use String? (Yes, in Java 7+)
4. What happens if break is missing?

---

15. Conclusion

Conditional statements are the backbone of programming logic. They allow programs to think, decide, and act dynamically.

Mastering these concepts will help in:

- Problem-solving
- Building real-world applications
- Writing efficient code

---

16. Final Thoughts (LinkedIn Style)

Today I learned how Java makes decisions using conditional statements. This topic strengthened my logic-building skills and helped me understand how real-world problems are solved in programming.

Consistency is key 🚀